



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان آزمایش:

طراحی و پیاده‌سازی مقایسه‌کننده‌های ترکیبی و ترتیبی (آبشاری و سریال)

نام درس

آزمایشگاه طراحی سیستم‌های دیجیتال

نیم‌سال ۱۴۰۳-۱۴۰۴

استاد

دکتر اجلالی

اسامی اعضای گروه

علی سلطانی - ۴۰۳۱۰۶۰۹۲

یگانه کربلایی - ۴۰۳۱۰۶۵۲۷

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	هدف آزمایش	۲
۲	مقایسه‌کننده ۴-بیتی آبشاری (Cascadable 4-Bit Comparator)	۲
۱.۲	توصیف مقایسه‌کننده ۱-بیتی آبشاری	۲
۲.۲	طراحی سلسله‌مراتبی مقایسه‌کننده ۴-بیتی	۳
۳.۲	سنتز و دیاگرام RTL در Quartus	۳
۴.۲	نتایج شبیه‌سازی در ModelSim و تحلیل عملکرد	۴
۳	مقایسه‌کننده سریال (Serial Comparator)	۴
۱.۳	توصیف و پیاده‌سازی مدار	۴
۲.۳	تحلیل نحوه پیاده‌سازی فلیپ‌فلاپ Master-Slave بدون بلوک always	۵
۳.۳	سنتز و دیاگرام RTL در Quartus	۶
۴.۳	نتایج شبیه‌سازی در ModelSim و تحلیل عملکرد	۶
۴	مقایسه منابع سخت‌افزاری و نتیجه‌گیری	۶

۱ هدف آزمایش

هدف از این آزمایش، آشنایی با نحوه پیاده‌سازی مقایسه‌کننده‌های منطقی ترکیبی و ترتیبی در زبان توصیف سخت‌افزار Verilog است. در این راستا، با توصیف جریان داده (Dataflow) با استفاده از دستور assign، ساختار سلسله‌مراتبی (Hierarchical Design) و همچنین مدل‌سازی ترتیبی بدون استفاده از بلوک‌های always (با استفاده از بازخورد و لچ/فلیپ‌فلاپ در سطوح انتساب پیوسته) آشنا می‌شویم.

۲ مقایسه‌کننده ۴-بیتی آبشاری (Cascadable 4-Bit Comparator)

۱.۲ توصیف مقایسه‌کننده ۱-بیتی آبشاری

در مقایسه‌کننده آبشاری ۱-بیتی، ۵ ورودی و ۳ خروجی داریم:

• a و b: بیت‌های جاری مربوط به دو عدد ورودی.

• A_eq_B_in، A_lt_B_in، A_gt_B_in: سیگنال‌های وضعیت حاصل از مقایسه بیت‌های پرارزش‌تر (MSB).

• A_eq_B_out، A_lt_B_out، A_gt_B_out: سیگنال‌های خروجی وضعیت مقایسه تا بیت جاری.

معادلات منطقی حاکم بر مدار به شرح زیر است:

$$A_{gt_out} = A_{gt_in} \vee (A_{eq_in} \wedge A \wedge \bar{B}) \quad (1)$$

$$A_{lt_out} = A_{lt_in} \vee (A_{eq_in} \wedge \bar{A} \wedge B) \quad (2)$$

$$A_{eq_out} = A_{eq_in} \wedge (\overline{A \oplus B}) \quad (3)$$

کد Verilog این پیمان‌ه در ادامه آمده است:

```
C: > Users > Dotcom > Desktop > HW3_403106092_ali_soltani_403106527_yeganeh_karbalaee_403106681 > quartus >  
1  module Cascadable1BitComparator (  
2      input a,  
3      input b,  
4      input A_gt_B_in,  
5      input A_lt_B_in,  
6      input A_eq_B_in,  
7      output A_gt_B_out,  
8      output A_lt_B_out,  
9      output A_eq_B_out  
10 );  
11  
12  
13     assign A_gt_B_out = A_gt_B_in | (A_eq_B_in & a & ~b);  
14     assign A_lt_B_out = A_lt_B_in | (A_eq_B_in & ~a & b);  
15     assign A_eq_B_out = A_eq_B_in & ~(a ^ b);  
16  
17 endmodule
```

شکل ۱: کد Verilog مقایسه‌کننده ۱-بیتی آبشاری

۲.۲ طراحی سلسله‌مراتبی مقایسه‌کننده ۴-بیتی

با Instantiation چهار باره از پیمانه ۱-بیتی، یک مقایسه‌کننده ۴-بیتی می‌سازیم. ورودی‌های اولیه برای باارزش‌ترین بیت (MSB) به صورت $A = B$ در نظر گرفته می‌شوند ($A_{eq_in} = 1$ و بقیه ۰). خروجی‌های هر مرحله به ورودی‌های آشناری بیت کم‌ارزش‌تر بعدی متصل شده و خروجی مرحله آخر، (LSB) خروجی کل مدار را تشکیل می‌دهد.

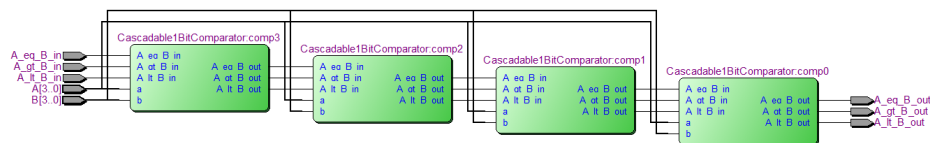
```
C: > Users > Dotcom > Desktop > HW3_403106092_ali_soltani_403106527_yeganeh_karbalaei_403106681 > quartus > CascadableComparator

1 module Cascadable4BitComparator (
2     input [3:0] A,
3     input [3:0] B,
4     input A_gt_B_in,
5     input A_lt_B_in,
6     input A_eq_B_in,
7     output A_gt_B_out,
8     output A_lt_B_out,
9     output A_eq_B_out
10 );
11
12 wire [3:0] w_gt;
13 wire [3:0] w_lt;
14 wire [3:0] w_eq;
15
16
17
18 Cascadable1BitComparator comp3 (
19     .a(A[3]), .b(B[3]),
20     .A_gt_B_in(A_gt_B_in), .A_lt_B_in(A_lt_B_in), .A_eq_B_in(A_eq_B_in),
21     .A_gt_B_out(w_gt[3]), .A_lt_B_out(w_lt[3]), .A_eq_B_out(w_eq[3])
22 );
23
24
25 Cascadable1BitComparator comp2 (
26     .a(A[2]), .b(B[2]),
27     .A_gt_B_in(w_gt[3]), .A_lt_B_in(w_lt[3]), .A_eq_B_in(w_eq[3]),
28     .A_gt_B_out(w_gt[2]), .A_lt_B_out(w_lt[2]), .A_eq_B_out(w_eq[2])
29 );
30
31
32 Cascadable1BitComparator comp1 (
33     .a(A[1]), .b(B[1]),
34     .A_gt_B_in(w_gt[2]), .A_lt_B_in(w_lt[2]), .A_eq_B_in(w_eq[2]),
35     .A_gt_B_out(w_gt[1]), .A_lt_B_out(w_lt[1]), .A_eq_B_out(w_eq[1])
36 );
37
38
39 Cascadable1BitComparator comp0 (
40     .a(A[0]), .b(B[0]),
41     .A_gt_B_in(w_gt[1]), .A_lt_B_in(w_lt[1]), .A_eq_B_in(w_eq[1]),
42     .A_gt_B_out(A_gt_B_out), .A_lt_B_out(A_lt_B_out), .A_eq_B_out(A_eq_B_out)
43 );
44
45 endmodule
```

شکل ۲: کد Verilog مقایسه‌کننده ۴-بیتی آشناری

۳.۲ سنتز و دیاگرام RTL در Quartus

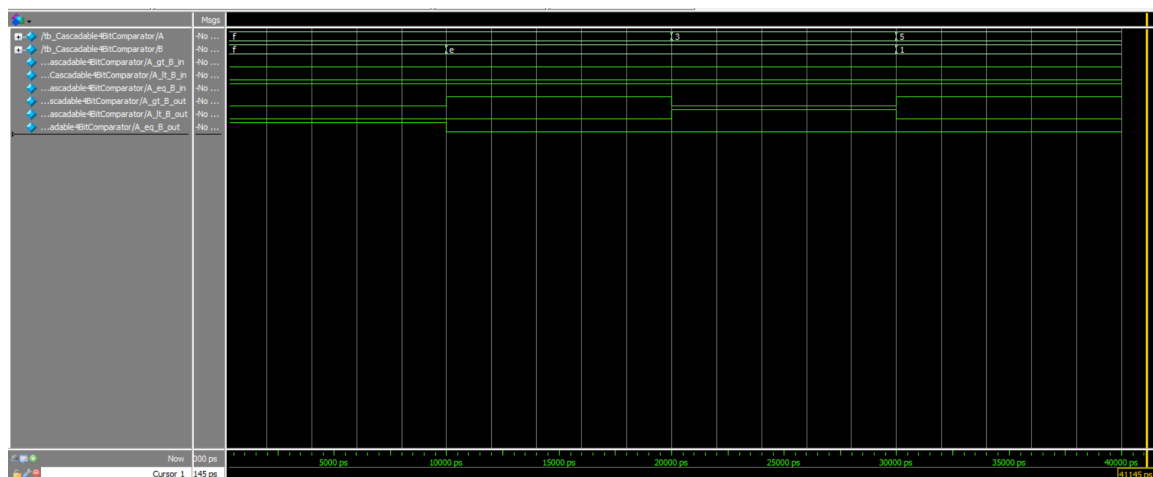
طرحواره RTL حاصل از سنتز مدار ۴-بیتی آشناری در نرم‌افزار Quartus نشان‌دهنده زنجیره‌سازی ۴ بلوک مقایسه‌کننده ۱-بیتی است.



شکل ۳: دیاگرام RTL مقایسه‌کننده ۴-بیتی آشناری در Quartus

۴.۲ نتایج شبیه‌سازی در ModelSim و تحلیل عملکرد

برای تست عملکرد مدار ترکیبی، حالات متفاوتی از ورودی‌های A و B اعم از تساوی، بزرگتر بودن و کوچکتر بودن اعمال شدند.



شکل ۴: موج‌فرم‌های شبیه‌سازی مقایسه‌کننده ۴-بیتی آشناری در ModelSim

تحلیل موج‌فرم شبیه‌سازی: همان‌طور که در شکل ۴ مشاهده می‌شود، به دلیل ترکیبی بودن ساختار مدار، به محض تغییر ورودی‌های A و B ، سیگنال‌های خروجی پس از گذر از تاخیر انتشار گیت‌ها (Propagation Delay) به‌روزرسانی می‌شوند:

- در بازه اولیه با اعمال $A = 15$ و $B = 15$ (حالت تساوی)، خروجی $A_eq_B_out$ برابر با ۱ باقی می‌ماند.
- با تغییر $A = 15$ و $B = 14$ ، از آنجا که بیت‌های MSB یکسان بوده اما در بیت LSB مقدار A بزرگتر است، سیگنال $A_gt_B_out$ بدون نیاز به سیگنال ساعت به سطح منطقی ۱ تغییر وضعیت می‌دهد.
- با تغییر $A = 3$ و $B = 14$ ، به دلیل بزرگتر بودن B در بیت‌های پرارزش، خروجی $A_lt_B_out$ فعال می‌شود.

۳ مقایسه‌کننده سریال (Serial Comparator)

۱.۳ توصیف و پیاده‌سازی مدار

مقایسه‌کننده سریال یک مدار ترتیبی است که در هر سیکل ساعت، یک جفت بیت از دو عدد را دریافت کرده و وضعیت مقایسه را به‌روزرسانی می‌کند.

کد کامل Verilog این مدار به شرح زیر است:

```

1 module SerialComparator (
2     input serial_a,
3     input serial_b,
4     input clk,
5     input rst,
6     output out_greater,
7     output out_equal,
8     output out_smaller
9 );
10
11 wire master_greater, prev_greater;
12 wire master_equal, prev_equal;
13 wire master_smaller, prev_smaller;
14
15
16 wire next_greater, next_equal, next_smaller;
17
18 assign next_greater = prev_greater | (prev_equal & serial_a & ~serial_b);
19 assign next_smaller = prev_smaller | (prev_equal & ~serial_a & serial_b);
20 assign next_equal = prev_equal & ~(serial_a ^ serial_b);
21
22
23 wire d_gt = rst ? 1'b0 : next_greater;
24 assign master_greater = clk ? d_gt : master_greater;
25 assign prev_greater = ~clk ? master_greater : prev_greater;
26
27
28 wire d_eq = rst ? 1'b1 : next_equal;
29 assign master_equal = clk ? d_eq : master_equal;
30 assign prev_equal = ~clk ? master_equal : prev_equal;
31
32
33 wire d_sm = rst ? 1'b0 : next_smaller;
34 assign master_smaller = clk ? d_sm : master_smaller;
35 assign prev_smaller = ~clk ? master_smaller : prev_smaller;
36
37
38 assign out_greater = prev_greater;
39 assign out_equal = prev_equal;
40 assign out_smaller = prev_smaller;
41
42 endmodule

```

شکل ۵: کد Verilog مقایسه‌کننده سریال با دستور assign

۲.۳ تحلیل نحوه پیاده‌سازی فلیپ‌فلاپ Master-Slave بدون بلوک always

در ساختارهای ترتیبی معمولاً از بلوک `(posedge always)` استفاده می‌شود؛ اما بر اساس صورت آزمایش، پیاده‌سازی صرفاً با دستورات انتساب پیوسته (`assign`) انجام گرفت. برای دستیابی به رفتار حساس به لبه (`Edge-Triggered`)، یک فلیپ‌فلاپ Master-Slave با استفاده از دو لچ و منطق بازخورد (`Combinational Loop`) طراحی شد:

$$\text{master} = \text{rst} ? 0 : (\text{clk} ? \text{master} : \text{cur}) \quad (۴)$$

$$\text{prev} = \text{rst} ? 0 : (\sim \text{clk} ? \text{master} : \text{prev}) \quad (۵)$$

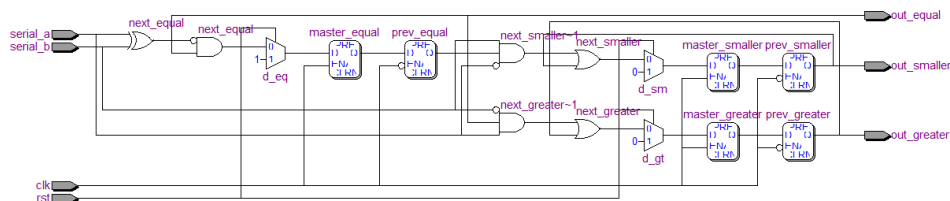
در این ساختار:

۱. هنگامی که $\text{clk} = 0$ است، لچ Master ورودی جدید (`cur`) را نمونه‌برداری کرده و شفاف (`Transparent`) است، در حالی که لچ Slave حالت قبلی را حفظ می‌کند.

۲. به محض صعود سیگنال ساعت ($\text{clk} = 1$)، لچ Master قفل شده و ارزش ورودی روی لچ Slave منتقل می‌شود. این سازوکار رفتاری کاملاً معادل با فلیپ‌فلاپ حساس به لبه بالا‌رونده ایجاد می‌کند.

۳.۳ سنتز و دیاگرام RTL در Quartus

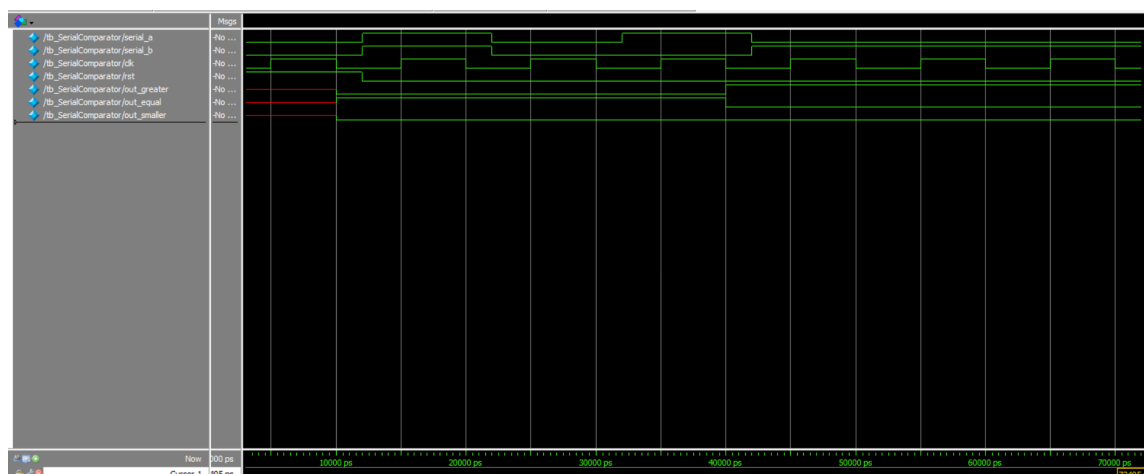
نتیجه سنتز مدار سریال در نرم افزار Quartus نشان دهنده ایجاد ساختار لچ ها و گیت های ترکیبی مربوط به منطق حالت بعدی است.



شکل ۶: دیاگرام RTL مقایسه کننده سریال در Quartus

۴.۳ نتایج شبیه سازی در ModelSim و تحلیل عملکرد

برای صحت سنجی مدار ترتیبی، داده های ورودی به صورت سری و بیت به بیت (MSB به LSB) همگام با لبه های ساعت اعمال گردیدند.



شکل ۷: موج فرم های شبیه سازی مقایسه کننده سریال در ModelSim

تحلیل موج فرم شبیه سازی: مطابق موج فرم های شکل ۷:

- با فعال شدن سیگنال rst، تمام خروجی ها بازنشانی شده و سیگنال out_equal به ارزش ۱ مقداردهی اولیه می شود.
- تا زمانی که بیت های ورودی serial_a و serial_b برابر باشند، وضعیت out_equal = ۱ حفظ می شود.
- با وقوع اولین عدم تطابق در یک لبه فعال ساعت (مثلاً serial_a = ۱ و serial_b = ۰)، خروجی out_greater فعال شده و به دلیل منطق قفل شونده (Latching Logic)، حتی با تغییرات بعدی ورودی ها در سیکل های بعد، وضعیت تا اعمال مجدد rst تغییر نمی کند.

۴ مقایسه منابع سخت افزاری و نتیجه گیری

بر اساس گزارش سنتز نرم افزار Quartus، میزان مصرف منابع سخت افزاری و ویژگی های عملکردی دو مدار در جدول زیر خلاصه سازی شده است:

جدول ۱: مقایسه عملکردی و منابع سخت‌افزاری دو نوع مقایسه‌کننده

شاخص مقایسه	مقایسه‌کننده ۴-بیتی آبشاری	مقایسه‌کننده سریال
عناصر منطقی (Logic Elements / LUTs)	رشد خطی متناسب با بیت‌ها ($O(N)$)	مقدار ثابت (مستقل از N)
ثبات‌ها/لج‌ها (Registers / Latches)	0	6 عنصر حافظه Master-Slave
تعداد سیکل محاسبه	1 سیکل ترکیبی	N سیکل ساعت
پیچیدگی سیم‌کشی (Routing)	بالا (در بیت‌های زیاد)	بسیار کم (ورودی‌های تک‌بیتی)

در نهایت، مقایسه‌کننده ترکیبی برای کاربردهایی که سرعت کارکرد بالا اولویت دارد و تعداد بیت‌ها محدود است مناسب بوده، در حالی که مقایسه‌کننده سریال برای سیستم‌های کم‌مصرف با محدودیت شدید مساحت چیپ (Silicon Area) گزینه ایده‌آل محسوب می‌شود.